

15. MANUAL TECNICO

SISTEMA DE GESTION PARA RESTAURANTE – SABOR & GESTION

Manual tecnico integral de instalacion, configuracion, ejecucion, administracion y resolucion de problemas del backend para la reproduccion determinista del entorno

FICHA METODOLOGICA DEL PROYECTO

Carrera:	Licenciatura en Ingenieria de Sistemas / Ingenieria Informatica
Asignatura:	Taller de Ingenieria de Software / Redes y Sistemas Distribuidos
Modulo Oficial:	Manual Tecnico de Administracion y Puesta en Marcha (Backend)
Arquitectura:	3 Capas Desacopladas (Routes -> Controllers -> Services -> Repositories)
Repositorio:	https://github.com/BandaTellezFernando/sabor-gestion-backend-personal
Version / Fecha:	Version 1.0.0 (Fases 1 a 17.6) — Octubre 2026
Publico Objetivo:	Estudiantes y evaluadores externos para reproduccion autonoma

STACK TECNOLÓGICO BASE:

Node.js >= 20.0.0 LTS • pnpm 12.4.2 • Express 5.2.1 • TypeScript 5.8+ • MongoDB Atlas • Socket.IO 4.8.3 • Cloudinary • EmailJS

TABLA DE CONTENIDOS

15.1	Requisitos del Sistema	3
15.1.1	Requisitos de Hardware	3
15.1.2	Requisitos de Software	3
15.2	Instalación Paso a Paso	4
15.3	Configuración del Sistema	8
15.3.1	Variables de Entorno (.env)	8
15.3.2	Base de Datos y Persistencia	8
15.3.3	Servicio de Generación de Códigos QR	9
15.3.4	Configuración de Puertos y CORS	9
15.4	Ejecución del Sistema	9
15.4.1	Modo Desarrollo (Hot Reload)	9
15.4.2	Compilación del Proyecto (Build)	9
15.4.3	Modo Producción	9
15.4.4	Comprobación del Funcionamiento (Healthcheck)	9
15.5	Administración del Sistema	10
15.5.1	Cómo Detener el Servidor	10
15.5.2	Cómo Reiniciar el Backend	10
15.5.3	Procedimiento de Actualización de Versión	10
15.5.4	Revisión y Monitoreo de Logs	10
15.5.5	Verificación de Servicios Vinculados	10
15.6	Solución de Problemas Frecuentes	11
15.7	Estructura y Organización del Código	12
15.8	Credenciales y Recomendaciones de Seguridad	13
15.9	Lista de Verificación Post-Instalación	13
15.10	Notas para el Desarrollador	13

Propósito y Alcance del Manual Técnico

Este documento constituye la memoria técnica formal de la sección "15. MANUAL TÉCNICO". Ha sido redactado para permitir que cualquier estudiante o evaluador técnico ajeno al equipo desarrollador pueda clonar, configurar, ejecutar, administrar y diagnosticar el backend de Sabor & Gestión sin requerir asistencia presencial ni inventar configuraciones inexistentes.

15.1 Requisitos del Sistema

Esta sección describe de manera formal y cuantitativa los recursos de hardware y componentes de software indispensables para ejecutar, compilar y operar el backend del proyecto Sabor & Gestión. Las especificaciones han sido determinadas a partir de las demandas reales del motor Node.js, la pila de dependencias en TypeScript, la concurrencia de WebSockets y la persistencia en MongoDB Atlas.

15.1.1 Requisitos de Hardware

El backend está diseñado bajo una arquitectura ligera orientada a servicios desacoplados en Node.js, lo que permite su ejecución tanto en estaciones de trabajo de desarrollo como en instancias virtuales de nube (VPS / PaaS).

Componente	Mínimo (Desarrollo)	Recomendado (Producción)	Justificación Técnica
Procesador (CPU)	2 núcleos (x86_64 o ARM64) a 2.0 GHz	4 núcleos a 2.5 GHz o superior	Node.js opera en un único hilo para el bucle de eventos (Event Loop), pero el compilador de TypeScript (tsc), el transpilador tsx, la librería criptográfica bcryptjs (cálculo de 10 rondas de salt) y la compresión demandan worker threads de libuv.
Memoria RAM	2.0 GB disponible	4.0 GB o superior	El proceso Node.js en ejecución estándar consume entre 80 MB y 180 MB. Durante la compilación (pnpm run build) o recarga en caliente con Nodemon, el compilador TypeScript almacena el árbol AST alcanzando picos de 600 MB a 1.2 GB.
Almacenamiento	1.5 GB espacio libre	5.0 GB almacenamiento SSD	El código fuente ocupa menos de 10 MB. El árbol de dependencias en node_modules requiere ~300-400 MB, la salida dist/ demanda 15 MB, y pnpm utiliza un almacén global indexado (hard links) para optimizar espacio.
Conexión de Red	Banda ancha estable (>= 2 Mbps)	Conexión simétrica baja latencia (< 50ms)	Estrictamente obligatoria. El backend conecta con MongoDB Atlas (puerto 27017 TCP), Cloudinary (puerto 443 HTTPS), EmailJS (puerto 443 HTTPS) y QR Server (puerto 443 HTTPS).

Nota sobre Periféricos y Sensores

Este backend no requiere aceleradores gráficos (GPU), dispositivos de captura biométrica ni periféricos especiales. Toda la interacción se realiza mediante protocolos HTTP/REST y WebSockets sobre TCP/IP.

15.1.2 Requisitos de Software

A continuación se detallan las plataformas, utilitarios y motores de ejecución requeridos para el funcionamiento del repositorio.

1. Sistemas Operativos Compatibles:
- Linux:** Distribuciones modernas basadas en Debian/Ubuntu (Ubuntu 20.04+, Debian 11+), Fedora 36+, Arch Linux o Alpine Linux (glibc).
 - macOS:** macOS Monterey (versión 12) o posterior (plataformas Intel o Apple Silicon M1/M2/M3/M4).
 - Windows:** Windows 10 (versión 2004+) o Windows 11 de 64 bits. Se recomienda encarecidamente el uso de WSL2 (Windows Subsystem for Linux) o PowerShell moderna con permisos adecuados.
2. Herramientas y Versiones Específicas del Proyecto:

Herramienta	Propósito en el Proyecto	Versión Requerida	Comando de Comprobación	Salida Típica
Git	Clonación del repositorio y sincronización de versiones.	>= 2.30.0	git --version	git version 2.43.0
Node.js	Entorno de ejecución JavaScript V8 del lado del servidor.	>= 20.0.0 LTS (v20, v22, v24)	node --version	v24.18.0 o v20.19.0
pnpm	Gestor de paquetes determinista según pnpm-lock.yaml.	>= 9.0.0 (Def: 12.4.2)	pnpm --version	12.4.2
TypeScript / tsc	Compilador estático oficial de TypeScript a JavaScript.	^5.0.0 (Act: 5.9.3)	npx tsc --version	Version 5.9.3
MongoDB Atlas	Persistencia NoSQL documental multiregión en la nube.	>= 6.0 (M0 Sandbox o sup.)	Conectividad SRV	mongodb+srv://... OK
Navegador Web	Inspección visual del healthcheck y URLs multimedia.	Cualquiera moderno	N/A	JSON de /api/health
Cliente HTTP	Emisión de comandas, multipart/ form-data y tokens Bearer.	Postman v10+, cURL, etc.	curl --version	curl 8.5.0 ...

15.2 Instalación Paso a Paso

El siguiente procedimiento describe de forma determinista la puesta en marcha desde una estación de trabajo limpia. El objetivo es que cualquier estudiante ajeno al grupo pueda replicar el entorno sin dudas técnicas.

Paso 1: Obtener el Código Fuente y Clonar el Repositorio

Abra una ventana de terminal y ejecute el comando de clonación indicando la URL oficial del repositorio:

Paso 1 — Clonar Repositorio Git

```
$ git clone https://github.com/BandaTellezFernando/sabor-gestion-backend-personal.git
Cloning into 'sabor-gestion-backend-personal'...
remote: Enumerating objects: 1240, done.
remote: Counting objects: 100% (1240/1240), done.
Receiving objects: 100% (1240/1240), 1.20 MiB | 3.50 MiB/s, done.
Resolving deltas: 100% (720/720), done.
```

- **Qué se hace:** Se descarga la copia exacta del repositorio oficial desde GitHub en el disco local.
- **Por qué se hace:** Es el paso inicial obligatorio para disponer del código fuente, historial y manifiestos de configuración.
- **Resultado esperado:** Directorio sabor-gestion-backend-personal creado con el árbol de código fuente íntegro.

Paso 2: Acceder al Directorio de Trabajo

Ingresa a la carpeta del proyecto recién descargada:

Paso 2 — Acceso al Directorio Raíz

```
$ cd sabor-gestion-backend-personal
```

- **Qué se hace:** Se establece el directorio actual en la raíz del proyecto backend.
- **Por qué se hace:** Todos los comandos de pnpm, variables de entorno y compilación deben ejecutarse desde la raíz.
- **Resultado esperado:** El prompt de la terminal indica la ruta ../sabor-gestion-backend-personal\$.

Paso 3: Verificar la Instalación de Node.js

Compruebe que el motor Node.js se encuentra instalado y cumple la versión mínima:

Paso 3 — Comprobación de Node.js

```
$ node --version
v24.18.0
```

- **Qué se hace:** Se solicita al sistema la versión del binario de Node.js.
- **Por qué se hace:** Versiones inferiores a v20.0.0 carecen de compatibilidad con características modernas de TypeScript y Express 5.
- **Resultado esperado:** Salida indicando v20.x, v22.x o v24.x.

Paso 4: Verificar o Instalar el Gestor de Paquetes pnpm

Verifique la disponibilidad de pnpm. Si no está instalado, actívelo con corepack o npm:

Paso 4 — Verificación e Instalación de pnpm

```
# Comprobación:
$ pnpm --version
12.4.2

# Instalación si no estuviera disponible:
$ corepack enable
$ corepack prepare pnpm@12.4.2 --activate
# Alternativa: npm install -g pnpm
```

- **Qué se hace:** Se comprueba y activa el gestor de paquetes oficial del proyecto.
- **Por qué se hace:** pnpm-lock.yaml requiere pnpm para garantizar la resolución determinista de dependencias sin duplicación.
- **Resultado esperado:** Versión 12.4.2 o superior desplegada.

Paso 5: Instalar Dependencias del Proyecto

Descargue e instale el árbol completo de librerías de producción y desarrollo:

Paso 5 — Instalación Determinista de Librerías

```
$ pnpm install
Resolving: total 27 dependencies, 11 devDependencies
Lockfile is up to date, resolution step is skipped
Packages: +258
Progress: resolved 258, reused 258, downloaded 0, added 258, done
Done in 3.2s
```

- **Qué se hace:** pnpm descarga los 258 paquetes declarados en pnpm-lock.yaml y construye node_modules.
- **Por qué se hace:** Provee Express 5, Mongoose 9, Socket.IO 4.8, jsonwebtoken, bcryptjs, Cloudinary, Multer, etc.
- **Resultado esperado:** Carpeta node_modules creada exitosamente con código de salida 0.

Paso 6: Crear el Archivo de Variables de Entorno (.env)

Cree el archivo de configuración local en la raíz del repositorio:

Paso 6 — Creación del Archivo .env

```
$ touch .env
```

- **Qué se hace:** Se genera el archivo de texto plano donde se definirán las claves y URLs de conexión.
- **Por qué se hace:** dotenv lee este archivo al iniciar el servidor para inyectar valores en process.env.
- **Resultado esperado:** Archivo .env presente en la raíz del repositorio.

Paso 7: Configuración de Clúster en MongoDB Atlas

Siga los pasos de aprovisionamiento en la nube de MongoDB Atlas:

- **1. Crear Clúster:** Registrarse en mongodb.com/cloud/atlas y desplegar un clúster gratuito M0 Sandbox.
- **2. Usuario de BD:** En Security -> Database Access, crear un usuario con rol Read and write to any database.
- **3. Acceso de Red:** En Security -> Network Access, agregar la IP actual o 0.0.0.0/0 (acceso global para desarrollo).
- **4. Cadena SRV:** En Connect -> Drivers, copiar la URI con protocolo mongodb+srv:// e insertarla en MONGO_URI en el .env.

Paso 8: Auditar la Integridad de las Dependencias Instaladas

Compruebe que los paquetes de nivel superior se encuentren correctamente indexados:

Paso 8 — Auditoría de Dependencias de Nivel Superior

```
$ pnpm list --depth=0
Legend: production dependency, optional dependency, dev dependency
sabor-gestion-backend@1.0.0 /home/.../sabor-gestion-backend-personal

dependencies:
bcryptjs 3.0.3      dotenv 17.3.1      mongoose 9.3.1      socket.io 4.8.3
cloudinary 2.9.0    express 5.2.1      morgan 1.10.1
cors 2.8.6          jsonwebtoken 9.0.3  multer 2.1.1
```

Paso 9: Carga Inicial de Datos Maestros (Seed)

Inicialice la base de datos con los datos maestros indispensables para operar:

Paso 9 — Ejecución del Script Seed

```
$ pnpm run seed
[OK] Conectado a la Base de Datos para seeding...
[OK] Colecciones limpias
[OK] 5 categorías creadas
[OK] Usuario Admin creado: admin@sabor.com / admin123
[OK] Usuario Mesero creado: mesero@sabor.com / mesero123
[OK] 2 platos de ejemplo creados
[OK] 8 mesas creadas

[OK] Seeding completado exitosamente
```

.

Qué se hace: Ejecuta `ts-node src/utls/seed.ts`. Limpia colecciones previas y genera categorías, platos, mesas y usuarios base.

- **Por qué se hace:** Permite que el evaluador pruebe de inmediato el login y la emisión de comandas sin crear datos a mano.
- **Credenciales creadas:** Admin: `admin@sabor.com / admin123` • Mesero: `mesero@sabor.com / mesero123`.

Paso 10: Carga de Datos de Inventario y Recetas (Opcional)

Para habilitar el control preventivo de ingredientes al emitir comandas:

Paso 10 — Carga de Inventario Gastronómico

```
$ pnpm run seed:inventario
[OK] Conectado a MongoDB para seed de inventario...
[OK] Colecciones Ingrediente y Receta limpiadas
[OK] Ingrediente creado: "Carne de Res" (disponible: true)
[OK] Ingrediente creado: "Papa" (disponible: true)
[OK] Ingrediente creado: "Huevo" (disponible: true)
[OK] Receta creada exitosamente vinculada al plato
```

15.3 Configuración del Sistema

15.3.1 Variables de Entorno (.env)

El backend utiliza dotenv para inyectar configuraciones en tiempo de ejecución. La siguiente tabla documenta de manera exhaustiva las 11 variables reales que consume el código fuente en src/:

Variable	Dominio	Oblig.	Valor por Defecto	Ejemplo Ficticio / Propósito
PORT	Servidor	No	3000	3000 — Puerto TCP de red del servidor HTTP y WebSockets.
NODE_ENV	Entorno	Sí	N/A	development — Protege contra ejecución de seed en producción.
MONGO_URI	Persistencia	Sí	N/A	mongodb+srv://usr:pwd@cluster.mongodb.net/sabor_db — Conexión Atlas.
JWT_SECRET	Seguridad	Sí	N/A	clave_secreta_jwt_2026 — Firma y verificación de tokens JWT.
CLOUDINARY_CLOUD_NAME	Multimedia	Sí*	N/A	mi-cloud — Nombre del entorno cloud en Cloudinary.
CLOUDINARY_API_KEY	Multimedia	Sí*	N/A	987654321012345 — Clave pública de la API de Cloudinary.
CLOUDINARY_API_SECRET	Multimedia	Sí*	N/A	abcdef1234567890abcdef — Llave secreta privada de Cloudinary.
EMAILJS_SERVICE_ID	Correo	Sí*	N/A	service_sabor — Service ID en EmailJS para recibos.
EMAILJS_TEMPLATE_ID	Correo	Sí*	N/A	template_recibo — ID de plantilla de correo transaccional.
EMAILJS_USER_ID	Correo	Sí*	N/A	user_public_key_emailjs — Clave pública de cuenta EmailJS.
EMAILJS_ACCESS_TOKEN	Correo	Sí*	N/A	token_privado_emailjs — Access Token para API REST EmailJS.

¡ATENCIÓN! DISCREPANCIA CRÍTICA DE NOMBRADO: MONGO_URI vs MONGODB_URI

En este repositorio, la variable de conexión a la base de datos se llama estrictamente MONGO_URI (src/configs/db.ts:8). Si se define por error como MONGODB_URI, el servidor lanzará: "Error: La variable de entorno MONGO_URI no está definida." y abortará con código 1. Utilice siempre MONGO_URI.

Plantilla de configuración de prueba (.env):

Plantilla Recomendada .env

```
PORT=3000
NODE_ENV=development

# Persistencia de Datos (MongoDB Atlas)
MONGO_URI=mongodb+srv://usuario_prueba:password123@cluster0.abcde.mongodb.net/sabor_gestion_db?
retryWrites=true&w=majority

# Criptografía y Sesiones Stateless (JWT)
JWT_SECRET=super_secret_jwt_key_sabor_gestion_2026_universidad

# Almacenamiento Multimedia de Platos (Cloudinary)
CLOUDINARY_CLOUD_NAME=demo_cloud_name
CLOUDINARY_API_KEY=123456789012345
CLOUDINARY_API_SECRET=abcdefghijklmnpqrstuvwxy123

# Despacho de Recibos por Correo Electrónico (EmailJS)
EMAILJS_SERVICE_ID=service_sabor_demo
EMAILJS_TEMPLATE_ID=template_recibo_demo
EMAILJS_USER_ID=user_public_key_demo
EMAILJS_ACCESS_TOKEN=token_privado_demo_emailjs
```

15.3.2 Base de Datos y Persistencia

- **Motor:** MongoDB Atlas (motor NoSQL documental multiregión).
- **Driver:** Mongoose v9.3.1 con esquemas estrictos y tipado TypeScript.
- **Cadena de Conexión:** Protocolo SRV seguro (mongodb+srv://). Incluye retryWrites=true y w=majority.
- **Restricciones de Red:** La dirección IP del host de ejecución debe encontrarse autorizada en Network Access de MongoDB

Atlas.

15.3.3 Servicio de Generación de Códigos QR

El proyecto no requiere librerías locales pesadas ni variables de entorno para generar códigos QR. En `src/services/pago.service.ts`, la función `generarPagoQR` construye dinámicamente un enlace que utiliza el servicio público `api.qrserver.com`:

Generación de QR vía Servicio Externo

```
const qrUrl = `https://api.qrserver.com/v1/create-qr-code/?size=250x250&data=${datosPago}`
```

Esto delega el renderizado de la matriz de puntos al CDN externo, ahorrando memoria y CPU en el servidor Node.js.

15.3.4 Configuración de Puertos y CORS

- **Puerto de Escucha:** Definido por `process.env.PORT` o 3000 por defecto si la variable no está configurada.
- **Orígenes CORS Permitidos:** Configurados de manera explícita en `src/app.ts` y `src/socket/socket.ts`: `http://localhost:5173`, `http://localhost:5174`, `https://quirquinita.onrender.com` y `https://tis-pied.vercel.app`.
- **Credenciales:** `credentials: true` habilitado para permitir intercambio seguro de cabeceras de autorización y cookies.

15.4 Ejecución del Sistema

El archivo `package.json` define tres comandos fundamentales de ejecución del servidor.

15.4.1 Modo Desarrollo (Hot Reload)

Para desarrollar de manera interactiva con recarga automática ante cambios de código:

Ejecución en Modo Desarrollo

```
$ pnpm run dev
[nodemon] 3.1.14
[nodemon] starting `tsx src/server.ts`
[OK] Base de Datos MongoDB Conectada: sabor_gestion_db
[INFO] Servidor ejecutandose en http://localhost:3000
[INFO] Health check: http://localhost:3000/api/health
[INFO] Servidor con WebSockets en http://localhost:3000
```

- **Herramientas involucradas:** Nodemon 3.1.14 monitorea el sistema de archivos y `tsx` transpila TypeScript en memoria.
- **Comportamiento:** Cualquier cambio guardado en `src/**/*.ts` reinicia el servidor en menos de 500 ms.

15.4.2 Compilación del Proyecto (Build)

Para verificar tipos estáticos y compilar la aplicación a JavaScript nativo:

Compilación a Producción

```
$ pnpm run build
```

- **Qué hace:** Ejecuta `tsc` leyendo `tsconfig.json`. Transpila los archivos de `src/` y genera la salida en `dist/`.
- **Resultado esperado:** Finaliza silenciosamente con código 0. Si existen errores de tipos, `tsc` los reporta y aborta.

15.4.3 Modo Producción

Para ejecutar el servidor compilado con máxima eficiencia de memoria:

Ejecución en Modo Producción

```
$ pnpm start
[OK] Base de Datos MongoDB Conectada: sabor_gestion_db
[INFO] Servidor ejecutandose en http://localhost:3000
[INFO] Health check: http://localhost:3000/api/health
[INFO] Servidor con WebSockets en http://localhost:3000
```

- **Qué hace:** Ejecuta `node dist/server.js` directamente sobre el motor V8 sin sobrecarga de transpilación.

15.4.4 Comprobación del Funcionamiento (Healthcheck)

Verifique la disponibilidad del servicio mediante el endpoint público de verificación de salud:

Verificación de Salud de la API

```
$ curl -X GET http://localhost:3000/api/health
{
  "status": "success",
  "message": "API de Sabor & Gestión funcionando correctamente"
}
```

15.5 Administración del Sistema

15.5.1 Cómo Detener el Servidor

- **En terminal interactiva:** Presione Ctrl + C. El proceso recibe la señal SIGINT y cierra ordenadamente las conexiones.
- **Proceso en background o puerto ocupado:** Localice y elimine el proceso con: `npx kill-port 3000` o identificando el PID con `lsof -i :3000` seguido de `kill -9 <PID>`.

15.5.2 Cómo Reiniciar el Backend

- **En desarrollo:** Escriba `rs` en la consola de Nodemon para forzar un reinicio manual sin salir del proceso.
- **Tras modificar el .env:** Detenga el servidor con Ctrl + C y vuelva a ejecutar `pnpm run dev` para recargar las variables.
- **En producción:** Ejecute `pnpm run build && pnpm start`.

15.5.3 Procedimiento de Actualización de Versión

Para actualizar el servidor con los cambios enviados al repositorio remoto:

Procedimiento de Actualización de Versión

```
# 1. Descargar cambios del repositorio
$ git pull origin main

# 2. Sincronizar librerías de pnpm-lock.yaml
$ pnpm install

# 3. Compilar TypeScript actualizado a dist/
$ pnpm run build

# 4. Iniciar el servicio actualizado
$ pnpm start
```

15.5.4 Revisión y Monitoreo de Logs

El sistema canaliza sus registros directamente a la salida estándar (stdout y stderr) mediante morgan en formato dev:

- **Peticiones HTTP:** Imprime método, ruta, código de estado y latencia: `GET /api/platos 304 4.102 ms`.
- **Conexiones WebSockets:** Registra eventos con prefijos semánticos: `[SOCKET] Usuario conectado: <id> (Rol: Mesero)` o `[SOCKET] Usuario desconectado`.
- **Errores no controlados:** Se escriben en stderr con el stack trace completo.

15.5.5 Verificación de Servicios Vinculados

Servicio	Método de Comprobación	Diagnóstico de Falla
Backend HTTP	Petición GET /api/health. Retorna HTTP 200.	Si no responde, el proceso no está corriendo o el puerto está bloqueado.
MongoDB Atlas	Log de arranque: [OK] Base de Datos MongoDB Conectada.	Si falla, imprime [ERROR] conectando y termina con <code>process.exit(1)</code> .
WebSockets	Conectar Socket.IO con token en auth. Imprime [SOCKET] Usuario conectado.	Si el token falta o es inválido, rechaza con: [SOCKET] Socket rechazado.
Cloudinary	POST /api/upload con imagen en multipart/form-data.	Si las credenciales son erróneas, devuelve HTTP 500 con error de API.
EmailJS	POST /api/pagos/enviar-recibo/:pedidoId con JSON de correo.	Si faltan variables en .env, lanza error: Credenciales no configuradas.

15.6 Solución de Problemas Frecuentes

La siguiente matriz diagnóstica resume las contingencias técnicas reales y sus soluciones verificadas:

Problema / Síntoma	Causa Raíz Probable	Diagnóstico y Solución Paso a Paso
Cannot find module "..." al iniciar	node_modules ausente o desincronizado con pnpm-lock.yaml.	Diagnóstico: Revisar carpeta node_modules. Solución: Ejecutar pnpm install para restaurar librerías.
EADDRINUSE: address already in use :::3000	El puerto 3000 está atrapado por otra instancia del servidor.	Diagnóstico: lsof -i :3000. Solución: Finalizar el proceso con npx kill-port 3000 o cambiar PORT=3001 en .env.
Error: MONGO_URI no está definida	Archivo .env ausente o variable nombrada erróneamente MONGODB_URI.	Diagnóstico: Verificar archivo .env. Solución: Asegurar que el archivo se llame .env y use la clave MONGO_URI.
connection timed out (MongoDB)	IP pública del host bloqueada por Network Access de MongoDB Atlas.	Diagnóstico: Panel de Atlas -> Network Access. Solución: Agregar la IP actual o 0.0.0.0/0 (desarrollo).
bad auth : Authentication failed	Usuario o contraseña erróneos en la cadena MONGO_URI.	Diagnóstico: Database Access en Atlas. Solución: Regenerar contraseña alfanumérica y actualizar cadena en .env.
JWT_SECRET no configurado	Variable JWT_SECRET ausente o vacía en el archivo .env.	Diagnóstico: grep JWT_SECRET .env. Solución: Agregar JWT_SECRET=clave_secreta_segura y reiniciar servidor.
tsc falla con errores TS2322 / TS2339	Inconsistencias de tipado en interfaces tras editar código en src/.	Diagnóstico: Ejecutar npx tsc --noEmit. Solución: Corregir los tipos señalados en la salida de terminal.
HTTP 401 Unauthorized en endpoints	Token no enviado en Authorization o token expirado (> 8 horas).	Diagnóstico: Inspeccionar cabeceras en Postman. Solución: Renovar token en POST /api/usuarios/login.
[SOCKET] Socket rechazado (sin token)	Cliente WebSocket conecta sin token en handshake auth.	Diagnóstico: Revisar logs del socket. Solución: Pasar { auth: { token } } al inicializar el cliente Socket.IO.
Seed no permitido en producción	NODE_ENV está configurado con valor production.	Diagnóstico: echo \$NODE_ENV. Solución: Configurar NODE_ENV=development en el .env antes de ejecutar el seed.

15.7 Estructura y Organización del Código

El repositorio implementa una estricta arquitectura en capas desacopladas, asegurando alta cohesión y testeabilidad:

Arbol de Directorios del Repositorio

```
sabor-gestion-backend-personal/
|-- .env                # Variables de entorno locales (ignorado por Git)
|-- .gitignore          # Reglas de exclusion de archivos confidenciales
|-- package.json        # Metadatos, dependencias y scripts oficiales
|-- pnpm-lock.yaml      # Arbol determinista de dependencias congeladas
|-- tsconfig.json       # Configuración del compilador oficial TypeScript
|-- vercel.json         # Manifiesto de despliegue serverless para Vercel
|-- docs/              # Documentación técnica formal y openapi.yaml
\-- src/
    |-- server.ts       # Punto de entrada HTTP y WebSockets
    |-- app.ts          # Express, middlewares globales y montaje de rutas
    |-- configs/        # Conexiones a servicios externos (db.ts, cloudinary.ts)
    |-- controllers/    # Controladores HTTP (manejo de req, res y códigos HTTP)
    |-- middlewares/    # Interceptores de seguridad (JWT, control de roles RBAC)
    |-- models/         # Esquemas Mongoose con validación de tipos y enums
    |-- repositories/   # Abstracción de acceso a datos y consultas a colecciones
    |-- routes/         # Enrutadores modulares por dominio de negocio
    |-- services/       # Capa de lógica de negocio y reglas transaccionales
    |-- socket/         # Gestión de WebSockets, salas y autenticación handshake
    |-- types/          # Definiciones de tipos, interfaces y DTOs
    \-- utils/          # Scripts de datos maestros (seed.ts, seedInventario.ts)
```

Flujo de Datos Arquitectónico

Flujo en Capas de la Petición

```
Peticion Cliente (HTTP / WebSocket)
|
v
[src/routes] -----> Define la URL, verbo HTTP y asocia middlewares.
|
v
[src/middlewares] -----> Valida el token JWT (verificarToken) y permisos (permitirRoles).
|
v
[src/controllers] -----> Extrae parametros de req, coordina respuesta y captura errores.
|
v
[src/services] -----> Ejecuta reglas de negocio (calcula de totales, stock, etc.).
|
v
[src/repositories] -----> Abstrae consultas a la base de datos (find, create, update).
|
v
[src/models] -----> Esquema Mongoose con validacion de tipos, enums e integridad.
|
v
[MongoDB Atlas] -----> Base de datos en la nube donde residen las colecciones BSON.
```

15.8 Credenciales y Recomendaciones de Seguridad

- **1. Protección del .env:** El archivo .env nunca debe incluirse en commits. El archivo .gitignore contiene explícitamente .env para prevenir fugas accidentales a repositorios públicos.
- **2. Entropía de JWT_SECRET:** En producción, utilice una clave criptográfica de al menos 32 caracteres generada aleatoriamente (ej. con openssl rand -base64 32).
- **3. Mínimo Privilegio en Atlas:** No use usuarios con rol atlasAdmin. Cree un usuario con permisos limitados a la base de datos sabor_gestion_db.
- **4. Protección Mass Assignment:** La capa de servicios filtra explícitamente los campos modificables, impidiendo la manipulación de estados financieros o totales.
- **5. Hashing de Contraseñas:** Las credenciales de usuarios se encriptan con bcryptjs utilizando 10 rondas de salt antes de persistirse.
- **6. Aislamiento de Entornos:** Nunca ejecute scripts de seed contra bases de datos en producción. NODE_ENV previene operaciones destructivas accidentales.

15.9 Lista de Verificación Post-Instalación

Utilice esta lista de control para certificar que el backend ha quedado correctamente desplegado y operativo:

- [] Node.js instalado en versión >= 20.0.0 (comprobado con node -v)
- [] pnpm instalado y reconocido en la terminal (comprobado con pnpm -v)
- [] Repositorio clonado e ingresado al directorio sabor-gestion-backend-personal
- [] Dependencias instaladas exitosamente mediante pnpm install
- [] Archivo .env creado con variables MONGO_URI y JWT_SECRET correctamente definidas
- [] IP pública autorizada en Network Access de MongoDB Atlas
- [] Datos iniciales cargados mediante pnpm run seed
- [] Servidor arrancado en desarrollo (pnpm run dev) o producción (pnpm start)
- [] Mensaje en consola confirma: [OK] Base de Datos MongoDB Conectada: sabor_gestion_db
- [] Endpoint GET /api/health retorna código HTTP 200 con mensaje de éxito
- [] Petición POST /api/usuarios/login con admin@sabor.com retorna token JWT válido
- [] Endpoint protegido GET /api/categorias con token Bearer responde HTTP 200
- [] Cliente WebSockets conecta a ws://localhost:3000 pasando el token en el handshake

15.10 Notas para el Desarrollador

Información de referencia para futuros ingenieros de software o evaluadores que extiendan el sistema:

- **Contrato OpenAPI Oficial:** La especificación de la API se encuentra en docs/openapi.yaml (OpenAPI 3.0.3). Puede visualizarse en Swagger Editor.
- **Extensión de Endpoints:** Para crear una nueva ruta, agregue el modelo en models/, repositorio en repositories/, servicio en services/, controlador en controllers/ y vincule en routes/ y app.ts.
- **Actualización de Estados:** En repositorios Mongoose, use siempre findByIdAndUpdate con { runValidators: true } para hacer cumplir los enums del esquema.
- **WebSockets y Salas:** Los sockets se organizan en salas room:meseros y room:caja según el rol decodificado del JWT.
- **Control de Calidad:** Ejecute pnpm run lint para verificar el cumplimiento de reglas ESLint y pnpm run build para certificar cero errores de tipos.